

O Codificador Limpo:

Um Código de Conduta para Programadores Profissionais



Prefácio, Introdução e Capítulo 1



PET/ADS/IFSP

**Este material foi desenvolvido pelo grupo
PET/ADS do IFSP São Carlos**

Prefácio

Matthew Heusser

Desenvolvedores São Profissionais ou Apenas Técnicos?

Matthew Heusser (Matt) conta, no prefácio, uma experiência em que ele coordenou uma equipe de desenvolvimento e acreditava que estava lidando com profissionais.

Porém, ao fim do projeto, ele percebeu que a equipe de desenvolvimento não era vista como profissional, na verdade eles eram vistos apenas como técnicos.

Os desenvolvedores não resolviam os problemas, eles não eram independentes. Qualquer problema que aparecia era resolvido por Matt e seu colega Joe.

O Codificador Limpo mostra o caminho para que técnicos passem a ser profissionais também.

Introdução

Pré-requisito

Introdução (parte 1)

Robert Martin, com 17 anos, foi contratado como programador temporário em uma empresa chamada ASC. Sua primeira tarefa foi atualizar manuais antigos.



Na folha está escrito
“esta página foi
deixada em branco
intencionalmente”.

Depois, ele foi designado para fazer um programa simples que seria escrito manualmente e gravado em cartões perfurados.

Introdução (parte 2)

Seu programa não rodou, então ele pediu ajuda para o supervisor, que corrigiu o erro e dispensou o jovem no dia seguinte.

Alguns meses depois ele voltou à empresa, dessa vez contratado em tempo integral.

Então, ele recebeu a tarefa de desenvolver um sistema de contabilidade com mais dois colegas.

Os três demoraram oito meses para acabar o sistema, entregaram o projeto e pediram demissão de maneira rebelde.

Introdução (parte 3)

Com 19 anos e desempregado, ele logo percebeu que não tomou uma boa decisão.

Ele pediu desculpa e foi recontratado com um salário menor do que tinha antes.

Ele ficou mais 18 meses na empresa, mas dessa vez saiu de forma amigável, pois tinha uma proposta de emprego melhor.

Esse não foi o último erro da carreira do autor, na verdade foi só o primeiro; em diversas outras vezes ele cometeu outros erros em postos e lugares diferentes.

Capítulo 1:

Profissionalismo

Cuidado com o que você pede

Profissionalismo é uma característica que todos almejam ter, mas ser **profissional** exige **responsabilidade**.

É fácil não ser profissional, basta não assumir as próprias responsabilidades e deixar isso para outras pessoas.

Assumir o impacto dos erros e reconhecer que cada falha possui uma consequência real que deve ser corrigida constitui requisito essencial do profissionalismo.

Assumindo responsabilidade

O desenvolvedor é **responsável** pelo seu código/trabalho.

Entregar no prazo é o ideal, porém entregar um código não testado ou inacabado para cumprir o prazo pode trazer muitos problemas.

Mesmo que demande mais tempo, o código deve ser entregue **pronto** e **testado**. Caso surjam bugs, cabe ao autor se responsabilizar e corrigi-los.

Não cause danos ao funcionamento

Todo software **deve rodar corretamente** . Um profissional busca minimizar ao máximo os bugs em seu software, mesmo que seja quase impossível zerá-los.

Cometer erros é comum, mas não se pode cometer o mesmo erro várias vezes.

A taxa de erros deve se aproximar de zero no longo prazo; provavelmente nunca chegará no zero, mas deve chegar perto.

O QA não deve achar nada

O código não deve ser enviado para o QA com a expectativa de que os erros sejam identificados por ela.

Se houver dúvidas sobre o funcionamento de algum trecho, é provável que esse código seja defeituoso.

Antes do envio, é necessário **garantir** que o código esteja funcionando.

Erros podem aparecer, mas isso deve ser uma **surpresa** e não uma certeza!

Você precisa saber que funciona

O melhor jeito de saber se um código funciona é fazendo **testes**, TUDO deve ser testado.

Tendo em vista a importância dos testes, o planejamento deve começar definindo como o código será testado, evitando que se torne difícil de testar depois.

Se os testes forem muito demorados, automatizá-los pode ser uma boa saída; assim, é possível executar vários testes de unidades repetidas vezes, economizando tempo.

Teste de QA automatizado

Para alguns projetos mais robustos, testes rápidos e automatizados podem ser insuficientes.

Porém, é essencial para o desenvolvedor ter uma ferramenta que:

- analise uma funcionalidade;
- elucide a interferência de uma função no resto do sistema;
- seja rápida.

Dessa forma, os testes automatizados devem, na medida do possível, dizer que é muito provável que o sistema passe no QA.

Não cause danos à estrutura

A estrutura de um código deve permitir mudanças. Código com **baixa manutenibilidade** prejudica o futuro, gera **perda de tempo e dinheiro**.

Se é difícil fazer mudanças no código, talvez o ideal seja revisar sua estrutura.

Não se pode ter medo de fazer mudanças num código que funcione, pois o real perigo é deixar o software rígido e dificultar sua manutenção.

O código deve ser tratado como um escultor trata a argila, moldando-a e dando forma continuamente.

Conheça seu campo

Nossa área tem uma diversidade de conteúdos; e um bom profissional deve conhecer boa parte deles.

Pode parecer que, por nossa área crescer tão rápido, o ideal é estudar apenas as tecnologias do momento, deixando de lado o que não está popular.

Isso não está certo, pois, em geral, as novas tecnologias não acabam com as antigas, nem as substituem de um dia para o outro, na verdade, isso demora décadas às vezes.

É interessante conhecer até mesmo o que não é mais usado, não para virar um especialista, mas para saber pontos positivos, negativos e porque ficou ultrapassado.

Aprendizagem contínua

O fato da nossa indústria se desenvolver muito rápido, só reforça a necessidade de reciclagem, aprendizado e aprimoramento constante.

É essencial manter-se informado sobre inovações e novidades.

A exploração fora da área de especialidade contribui para acompanhar avanços em outros campos e encontrar novos desafios.

Sempre há algo novo para estudar ou pesquisar. Um bom desenvolvedor não para de buscar novos conhecimentos, pois sabe que sempre haverá algo que ele ainda desconhece.

Pratique

O desenvolvedor é responsável por sua própria carreira; portanto ele deve sempre estar buscando se aprimorar por meio da prática.

Para melhorar como desenvolvedor, é necessário sempre estar praticando. Prática é aquilo que é feito fora do trabalho, trabalho é performance.

prática -> treino

trabalho -> jogo

Uma boa maneira de praticar é fazendo exercícios simples de programação repetidas vezes para treinar habilidades (ou uma linguagem) que deseja melhorar.

Colaboração e ensino

É importante colaborar com os colegas, pois, um desenvolvedor pode estar tentando resolver algo que seu colega tem a solução e pode ajudá-lo.

Colaboração pode tornar o trabalho mais simples, rápido e fácil.

Uma ótima forma de aprender/consolidar um conhecimento é ensinando ele para outras pessoas.

Um bom profissional não deixa um novato sem supervisão, pois um ótimo jeito de recepcionar um novato é sendo um “professor” para ele.

Conheça seu domínio

É importante conhecer para quem se trabalha e qual é a área de atuação do cliente.

Se um cliente trabalha na área de contabilidade, busque entender sobre a área e sobre o que ele faz.

Não é necessário (nem possível) se tornar um especialista na área de atuação de todos os clientes, porém deve-se tentar entender o mínimo necessário.

Livros, artigos ou conversas com especialistas da área são boas fontes de informações para entender o contexto em que um software estará envolvido.

Identifique-se com seu empregador/cliente

É necessário compreender as necessidades do empregador ou cliente, colocando-se no lugar deles e estabelecendo diálogo para assegurar o alinhamento.

Dessa forma, as soluções desenvolvidas têm mais chances de atenderem às necessidades reais do empregador/cliente.

Deve-se evitar situações de conflito com o empregador; e isso não significa não discordar ou questionar, mas, sim, não criar um cenário de “Nós vs Eles”.

Humildade

Um profissional sabe que é falho e, quando erra, aceita que errou e aceita as críticas que isso vai acarretar.

Sabendo disso, ele jamais deixa de aceitar o erro de outras pessoas, ou as ridiculariza, pois sabe que o próximo a errar pode ser ele.

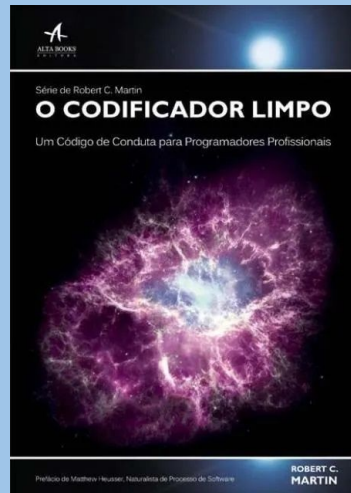
Reconhecer a própria imperfeição e aceitar críticas ou falhas sem ridicularizar os outros reforça a humildade do profissional.

Sobre o autor

Robert C. “Uncle Bob” Martin é desenvolvedor e consultor de software desde 1990. Ele é o fundador e o presidente da Object Mentor, Inc., uma equipe de consultores experientes que orientam seus clientes no mundo todo em C++, Java, C#, Ruby, OO, Padrões de Projeto, UML, Metodologias Agile e eXtreme Programming.

Este conjunto de slides foi elaborado a partir da obra:

MARTIN, Robert C. **O codificador limpo:**
Um código de conduta para programadores
profissionais. 1. ed. Rio de Janeiro: Alta
Books, 2012.



A equipe



Gabriel Ribeiro, Autor

Aluno do segundo período, é bolsista do PET/ADS desde novembro de 2025.



Gabriel Bocute, Revisor

Aluno do quarto período, é bolsista do PET/ADS desde março de 2025.



Lucas Oliveira, Tutor

Professor de Computação, é tutor do PET/ADS desde janeiro de 2023.